

Bitstring Status List v1.0

Privacy-preserving status information for Verifiable Credentials



W3C Recommendation 15 May 2025

▼ More details about this document

This version:

<https://www.w3.org/TR/2025/REC-vc-bitstring-status-list-20250515/>

Latest published version:

<https://www.w3.org/TR/vc-bitstring-status-list/>

Latest editor's draft:

<https://w3c.github.io/vc-bitstring-status-list/>

History:

<https://www.w3.org/standards/history/vc-bitstring-status-list/>

[Commit history](#)

Implementation report:

<https://w3c.github.io/vc-bitstring-status-list-test-suite/>

Editors:

[Manu Sporny](#) ([Digital Bazaar](#))

[Dave Longley](#) ([Digital Bazaar](#))

[Mike Prorock](#) ([mesur.io](#))

[Mahmoud Alkhraishi](#) ([Mavennet](#))

Authors:

[Dave Longley](#) ([Digital Bazaar](#))

[Manu Sporny](#) ([Digital Bazaar](#))

[Orie Steele](#) ([Transmute](#))

Feedback:

[GitHub w3c/vc-bitstring-status-list](#) ([pull requests](#), [new issue](#), [open issues](#))

Errata:

[Errata exists.](#)

Related Documents

[Verifiable Credentials Data Model v2.0](#)

See also [translations](#).

Copyright © 2025 World Wide Web Consortium. W3C[®] [liability](#), [trademark](#) and [permissive document license](#) rules apply.

Abstract

This specification describes a privacy-preserving, space-efficient, and high-performance mechanism for publishing status information such as suspension or revocation of Verifiable Credentials through use of bitstrings.

Status of This Document

This section describes the status of this document at the time of its publication. A list of current W3C publications and the latest revision of this technical report can be found in the [W3C standards and drafts index](https://www.w3.org/TR/) at <https://www.w3.org/TR/>.

Comments regarding this specification are welcome at any time. Please file issues directly on [GitHub](#), or send them to public-vc-comments@w3.org if that is not possible. ([subscribe](#), [archives](#)).

This document was published by the [Verifiable Credentials Working Group](#) as a Recommendation using the [Recommendation track](#).

W3C recommends the wide deployment of this specification as a standard for the Web.

A W3C Recommendation is a specification that, after extensive consensus-building, is endorsed by W3C and its Members, and has commitments from Working Group members to [royalty-free licensing](#) for implementations.

This document was produced by a group operating under the [W3C Patent Policy](#). W3C maintains a [public list of any patent disclosures](#) made in connection with the deliverables of the group; that page also includes instructions for disclosing a patent. An individual who has actual knowledge of a patent which the individual believes contains [Essential Claim\(s\)](#) must disclose the information in accordance with [section 6 of the W3C Patent Policy](#).

This document is governed by the [03 November 2023 W3C Process Document](#).

Table of Contents

Abstract

Status of This Document

- 1. Introduction**
 - 1.1 Conceptual Framework
 - 1.2 Terminology
 - 1.3 Conformance

2. Data Model

- 2.1 BitstringStatusListEntry
- 2.2 BitstringStatusListCredential

3. Algorithms

- 3.1 Generate Algorithm
- 3.2 Validate Algorithm
- 3.3 Bitstring Generation Algorithm
- 3.4 Bitstring Expansion Algorithm
- 3.5 Processing Errors
- 3.6 Securing Algorithms

4. Media Types**5. Contexts and Vocabularies**

- 5.1 Vocabulary
- 5.2 JSON-LD context

6. Privacy Considerations

- 6.1 Revocation Bitstring Length
- 6.2 Unnecessary Correlation
- 6.3 Verifier Caching
- 6.4 Content Distribution Networks
- 6.5 Decoy Values
- 6.6 Malicious Issuers and Verifiers
- 6.7 Monitoring Status Lists
- 6.8 Correlation of Status Messages
- 6.9 Alteration of Status Messages

7. Security Considerations

- 7.1 Bitstring Encoding
- 7.2 Validity Periods

8. Accessibility Considerations**9. Internationalization Considerations****A. Examples**

- A.1 Revocable Verifiable Credential
- A.2 Status List Verifiable Credential
- A.3 Multiple Status Lists in One Verifiable Credential
- A.4 Multiple Status Entries in a Single List

B. Revision History**C. References**

C.1 Normative references

C.2 Informative references

§ 1. Introduction

This section is non-normative.

It is often useful for an [issuer](#) of [verifiable credentials](#) [VC-DATA-MODEL-2.0] to link to a location where a [verifier](#) can check to see if a credential has been suspended or revoked. There are a variety of privacy and performance considerations that are made when designing, publishing, and processing status lists.

One such privacy consideration happens when there is a one-to-one mapping between a [verifiable credential](#) and a URL where the status is published. This type of mapping enables the website that publishes the URL to correlate the [holder](#), time, and [verifier](#) when the status is checked. This could enable the [issuer](#) to discover the type of interaction the [holder](#) is having with the [verifier](#), such as providing an age verification credential when entering a bar. Being tracked by the [issuer](#) of a driver's license when entering an establishment violates a privacy expectation that many people have today.

Similarly, there are performance considerations that are explored when designing status lists. One such consideration is where the list is published and the burden it places from a bandwidth and processing perspective, both on the server and the client fetching the information. In order to meet privacy expectations, it is useful to bundle the status of large sets of credentials into a single list to help with group privacy. However, doing so can place an impossible burden on both the server and client if the status information is as much as a few hundred bytes in size per credential across a population of hundreds of millions of [holders](#).

The rest of this document proposes a highly compressible, bitstring-based status list mechanism with strong privacy-preserving characteristics, that is compatible with the architecture of the Web, is highly space-efficient, and lends itself well to content distribution networks. As an example of using this specification to achieve a number of beneficial privacy and performance goals, it is possible to create a status list that can be constructed for 100,000 [verifiable credentials](#) that is roughly 12,500 bytes in size in the worst case. In a case where a few hundred credentials have been revoked, the size of the list is less than a few hundred bytes while providing privacy in a group of 100,000 individuals.

§ 1.1 Conceptual Framework

This section is non-normative.

This section outlines the core concept utilized by the status list mechanism described in this specification. At the most basic level, status information for all [verifiable credentials](#) issued by an [issuer](#) is expressed as items in a list. Each [issuer](#) manages a list of all [verifiable credentials](#) that it has issued. Each [verifiable credential](#) is associated with an item in its list. When a single bit specifies a status, such as "revoked" or "suspended", then that status is expected to be true when the bit is set (**1**) and false when unset (**0**).

One of the benefits of using a bitstring is that it is a highly compressible data format since, in the average case, large numbers of credentials will remain unrevoked. This will ensure long sections of bits that are the same value and thus highly compressible using run-length compression techniques such as GZIP [RFC1952]. The default status list size is 131,072 entries, equivalent to 16 KB of single bit values. When only a handful of [verifiable credentials](#) are revoked, GZIP compresses the bitstring to a few hundred bytes.

Another benefit of using a bitstring is that it enables large numbers of [verifiable credential](#) statuses to be placed in the same list. This specification uses a minimum list length of 131,072. This size ensures an adequate amount of group privacy in the average case. If better group privacy is required, the bitstring can be made larger.

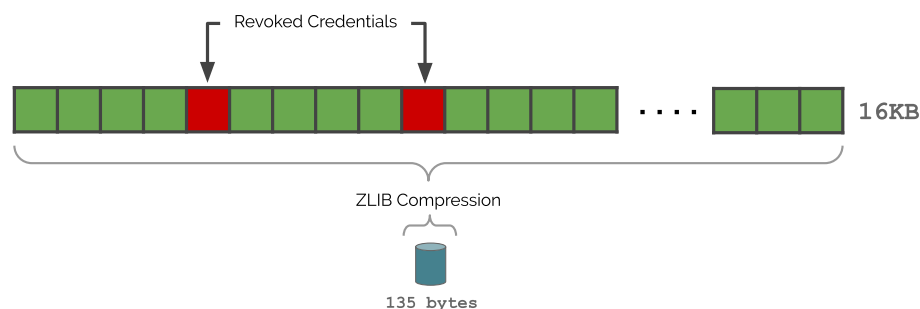


Figure 1 A visual depiction of the concepts outlined in this section.

NOTE: Status information is about the verifiable credential

The status information associated with a particular [verifiable credential](#) is about the [verifiable credential](#) itself and might not apply to any underlying or backing [credential](#), such as an educational degree. For example, in the case of such an educational degree, it is possible for a [verifiable credential](#) to be revoked because the mechanism used to create its digital signature has been compromised, while the backing educational degree remains valid.

§ 1.2 Terminology

This section is non-normative.

Terminology used throughout this document is defined in the [Terminology](#) section of the [Verifiable Credentials Data Model v2.0](#) specification.

§ 1.3 Conformance

As well as sections marked as non-normative, all authoring guidelines, diagrams, examples, and notes in this specification are non-normative. Everything else in this specification is normative.

The key words *MAY*, *MUST*, *MUST NOT*, *OPTIONAL*, *SHOULD*, and *SHOULD NOT* in this document are to be interpreted as described in [BCP 14](#) [[RFC2119](#)] [[RFC8174](#)] when, and only when, they appear in all capitals, as shown here.

A **conforming document** is any concrete expression of the data model that follows the relevant normative requirements in Section [2. Data Model](#).

A **conforming processor** is any algorithm realized as software and/or hardware that generates and/or consumes a [conforming document](#) according to the relevant normative statements in Section [3. Algorithms](#). Conforming processors *MAY* choose to only support bitstring entry sizes of 1. Conforming processors *MUST* produce errors when non-conforming documents are consumed.

§ 2. Data Model

There are numerous ways to express status information associated with digital credentials. Some of these mechanisms include Certificate Revocation Lists (CRL) [[RFC5280](#)], the Online Certificate Status Protocol (OCSP) [[RFC2560](#)], Bloom Filters [[RFC8932](#)], and cryptographic accumulators [[ALLOSAUR](#)]. This specification optimizes for a variety of requirements that are different from other mechanisms. These requirements include:

Comparison of Status Technologies					
Feature	CRL	OCSP	Bloom	Accumulator	Bitstring
Provides tunable group privacy	✓	✗	✓	✓	✓
Does not require signed assertion for each credential	✓	✗	✓	✓	✓

Comparison of Status Technologies					
Feature	CRL	OCSP	Bloom	Accumulator	Bitstring
Resistant to issuer tracking when fetched by verifier	✓	✓	✓	✓	✓
Caching is space efficient with many revocations	✗	✗	✓	✓	✓
Highly compressible (>90% average compression)	✗	✗	✓	✓	✓
Updates are efficient (fast and entire population does not need to update)	✓	✗	✓	✗	✓
Uses cryptographic primitives approved by IETF	✓	✓	✓	✗	✓
No false positives	✓	✓	✗	✓	✓
Can be delivered by holder (stapling)	✗	✓	✗	✓	✓
Easily profiled for usage with verifiable credentials	✗	✗	✗	✗	✓

§ 2.1 BitstringStatusListEntry

When an [issuer](#) desires to enable status information for a [verifiable credential](#), they *MAY* add a **credentialStatus** property that uses the data model described in this section. Any expression of the data model in this section *MUST* be expressed in a conforming [verifiable credential](#) as defined in [VC-DATA-MODEL-2.0].

Property	Description
id	An optional identifier for the status list entry. The constraints on the id property are listed in the Verifiable Credentials Data Model specification [VC-DATA-

Property	Description										
	<i>MODEL-2.0</i>]. If present, the value is expected to be a URL that identifies the status information associated with the verifiable credential. It <i>MUST NOT</i> be the URL for the status list. The value is not used during the verification or validation process, and does not need to be related to the statusListCredential value. If necessary, the value can be used to uniquely identify the BitstringStatusListEntry object, such as when it is stored in a database.										
type	The type property <i>MUST</i> be BitstringStatusListEntry .										
statusPurpose	The purpose of the status entry <i>MUST</i> be a string. While the value of the string is arbitrary, the following values <i>MUST</i> be used for their intended purpose: <table> <tr> <th>Value</th><th>Description</th></tr> <tr> <td>refresh</td><td>Used to signal that an updated verifiable credential is available via the credential's refresh service feature. This status does not invalidate the verifiable credential and is not reversible.</td></tr> <tr> <td>revocation</td><td>Used to cancel the validity of a verifiable credential. This status is not reversible.</td></tr> <tr> <td>suspension</td><td>Used to temporarily prevent the acceptance of a verifiable credential. This status is reversible.</td></tr> <tr> <td>message</td><td>Used to convey an arbitrary message related to the status of the verifiable credential.</td></tr> </table>	Value	Description	refresh	Used to signal that an updated verifiable credential is available via the credential's refresh service feature. This status does not invalidate the verifiable credential and is not reversible.	revocation	Used to cancel the validity of a verifiable credential . This status is not reversible.	suspension	Used to temporarily prevent the acceptance of a verifiable credential . This status is reversible.	message	Used to convey an arbitrary message related to the status of the verifiable credential .
Value	Description										
refresh	Used to signal that an updated verifiable credential is available via the credential's refresh service feature. This status does not invalidate the verifiable credential and is not reversible.										
revocation	Used to cancel the validity of a verifiable credential . This status is not reversible.										
suspension	Used to temporarily prevent the acceptance of a verifiable credential . This status is reversible.										
message	Used to convey an arbitrary message related to the status of the verifiable credential .										
statusListIndex	The statusListIndex property <i>MUST</i> be an arbitrary size integer greater than or equal to 0, expressed as a string in base 10. The value identifies the position of the status of the verifiable credential . Implementations <i>SHOULD</i> assign indexes randomly, such that inferences — such as the recency of the assignment or the size of the group — cannot be easily drawn from that position.										
statusListCredential	The statusListCredential property <i>MUST</i> be a URL to a verifiable credential . When the URL is dereferenced, the resulting verifiable credential										

Property	Description
	<i>MUST</i> have type property that includes the BitstringStatusListCredential value.
statusSize	The statusSize indicates the size of the status entry in bits. statusSize <i>MAY</i> be provided. If statusSize is not present as a property of the credentialStatus , then statusSize <i>MUST</i> be processed as 1 . If present, statusSize <i>MUST</i> be an integer greater than zero. If statusSize is provided and is greater than 1 , then the property credentialStatus.statusMessage <i>MUST</i> be present, and the number of status messages <i>MUST</i> equal the number of possible values.
statusMessage	If present, the statusMessage property <i>MUST</i> be an array, the length of which <i>MUST</i> equal the number of possible status messages indicated by statusSize (e.g., statusMessage array <i>MUST</i> have 2 elements if statusSize has 1 bit, 4 elements if statusSize has 2 bits, 8 elements if statusSize has 3 bits, etc.). statusMessage <i>MAY</i> be present if statusSize is 1, and <i>MUST</i> be present if statusSize is greater than 1. If the statusMessage array is not present, the message values associated with the status bit values of 1 and 0 are "set" and "unset", respectively. If the statusMessage array is present, each element <i>MUST</i> contain the two properties described below, and <i>MAY</i> contain additional properties. • status, a string representing the hexadecimal value of the status prefixed with 0x • message, a string used by software developers to assist with debugging which <i>SHOULD NOT</i> be displayed to end users. Implementers <i>MAY</i> add additional values to objects in the statusMessage array. Implementers <i>MAY</i> use the string value of undefined in the value to indicate that a corresponding status is not defined for the associated status value, but that it may be defined in the future. Rules for how to handle various status messages are outside the scope of normative requirements in this document, but it is assumed that implementers will document rules for processing various status codes.
statusReference	An implementer <i>MAY</i> include the statusReference property. If present, its value <i>MUST</i> be a URL or an array of URLs [URL] which dereference to material related to the status. Implementers using a statusPurpose of message are strongly encouraged to provide a statusReference .

Property	Description
	NOTE: Details around reference statusReference is especially important when interpretation of the status for a credential may involve some understanding of the business case involved.

Status list entries can be used to express the purpose of a status associated with a [verifiable credential](#) by using the **statusPurpose** property.

The use of **revocation** or **suspension** as the status purpose includes the semantics of the status, with **revocation** indicating that a status bit expresses whether a [verifiable credential](#) has been revoked and **suspension** indicating that a status bit expresses whether a [verifiable credential](#) has been suspended. The example below demonstrates the use of these status purposes:

EXAMPLE 1: Example StatusListCredential using simple entries

```

{
  "@context": [
    "https://www.w3.org/ns/credentials/v2",
    "https://www.w3.org/ns/credentials/examples/v2"
  ],
  "id": "https://example.com/credentials/23894672394",
  "type": ["VerifiableCredential", "EmployeeIdCredential"],
  "issuer": "did:example:12345",
  "validFrom": "2024-04-05T14:27:42Z",
  "credentialStatus": [{
    "id": "https://example.com/credentials/status/3#94567",
    "type": "BitstringStatusListEntry",
    "statusPurpose": "revocation",
    "statusListIndex": "94567",
    "statusListCredential": "https://example.com/credentials/status/3"
  }, {
    "id": "https://example.com/credentials/status/4#23452",
    "type": "BitstringStatusListEntry",
    "statusPurpose": "suspension",
    "statusListIndex": "23452",
    "statusListCredential": "https://example.com/credentials/status/4"
  }],
  "credentialSubject": {
    "id": "did:example:6789",
    "type": "Person",
    "employeeId": "A-123456"
  }
}

```

The use of **message** as the status purpose enables an issuer to define an arbitrary number of custom, descriptive messages about the status of the [verifiable credential](#). The [issuer](#) commits to the set of messages that may be associated with a particular entry (i.e., with a particular [verifiable credential](#)) through the **statusSize**, **statusMessage**, and optional **statusReference** properties, at the time of [verifiable credential](#) issuance. This is to ensure that the [holder](#) knows what sort of information might be associated with a particular [verifiable credential](#) they keep in their possession, that could then be discoverable by a [verifier](#) that later receives that credential.

NOTE

It is important to note that **statusListIndex** is the only link between the [verifiable credential](#) and its status in the list. Other properties such as **credentialSubject.id** are not used for this purpose.

EXAMPLE 2: Example StatusListCredential using more complex entries

```

{
  "@context": [
    "https://www.w3.org/ns/credentials/v2",
    "https://www.w3.org/ns/credentials/examples/v2"
  ],
  "id": "https://example.com/credentials/2947478373",
  "type": ["VerifiableCredential", "BillOfLadingExampleCredential"],
  "issuer": "did:example:12345",
  "validFrom": "2024-04-05T03:52:31Z",
  "credentialStatus": {
    "id": "https://example.com/credentials/status/8#492847",
    "type": "BitstringStatusListEntry",
    "statusPurpose": "message",
    "statusListIndex": "492847",
    "statusSize": 2,
    "statusListCredential": "https://example.com/credentials/status/8",
    "statusMessage": [
      {"status": "0x0", "message": "pending_review"},
      {"status": "0x1", "message": "accepted"},
      {"status": "0x2", "message": "rejected"},
      ...
    ],
    "statusReference": "https://example.org/status-dictionary/"
  },
  "credentialSubject": {
    "id": "did:example:6789",
    "type": "BillOfLading",
    ...
  }
}

```

§ 2.2 BitstringStatusListCredential

When a status list [verifiable credential](#) is published, it *MUST* be a conforming document, as defined in [VC-DATA-MODEL-2.0], that expresses the data model in this section. The following section describes the format of the [verifiable credential](#) that encapsulates the status list.

The status list is expressed inside a [verifiable credential](#) in order to enable a [holder](#) to provide it directly to a [verifier](#). This mechanism, sometimes called "certificate stapling", increases privacy for the [holder](#) by ensuring that the [verifier](#) does not need to contact the [issuer](#) to retrieve the status list. Still, a [verifier](#) might choose to

ignore the [holder](#)-provided status list, even when its authenticity is verifiable, if it desires a more recent version of a status list, for instance.

[Issuers](#) and [verifiers](#) are advised that the [issuer](#) of a [verifiable credential](#) and the [issuer](#) of an associated **BitstringStatusListCredential** might not be the same. There are technical, legal, institutional, political, and other reasons that might make it appropriate to separate the authority over the original credential from the authority to revoke, or otherwise change the status of, such a credential. Therefore, the **issuer** value of a [verifiable credential](#) containing a **BitstringStatusListEntry** *MAY* be different from the **issuer** value of a **BitstringStatusListCredential**.

Property	Description				
id	The verifiable credential that contains the status list <i>MAY</i> express an id property that matches the value specified in statusListCredential for the corresponding BitstringStatusListEntry (see 2.1 BitstringStatusListEntry).				
type	The verifiable credential that contains the status list <i>MUST</i> express a type property that includes the BitstringStatusListCredential value.				
validFrom	The earliest point in time at which the status list is valid. This property is defined in the Verifiable Credentials Data Model specification in Section 4.6: Validity Period .				
validUntil	The latest point in time at which the status list is valid. This property is defined in the Verifiable Credentials Data Model specification in Section 4.6: Validity Period .				
credentialSubject.type	The type of the credential subject , which is the status list, <i>MUST</i> be BitstringStatusList .				
credentialSubject.statusPurpose	The value of the purpose property of the status entry, statusPurpose, <i>MUST</i> be one or more strings. While the value of each string is arbitrary, the following values <i>MUST</i> be used for their intended purpose: <table> <tr> <th>Value</th><th>Description</th></tr> <tr> <td>refresh</td><td>Used to signal that an updated verifiable credential is available via the credential's</td></tr> </table>	Value	Description	refresh	Used to signal that an updated verifiable credential is available via the credential's
Value	Description				
refresh	Used to signal that an updated verifiable credential is available via the credential's				

Property	Description										
	<table> <tr> <th>Value</th><th>Description</th></tr> <tr> <td></td><td>refresh service feature. This status does not invalidate the verifiable credential and is not reversible.</td></tr> <tr> <td>revocation</td><td>Used to cancel the validity of a verifiable credential. This status is not reversible.</td></tr> <tr> <td>suspension</td><td>Used to temporarily prevent the acceptance of a verifiable credential. This status is reversible.</td></tr> <tr> <td>message</td><td>Used to indicate a status message associated with a verifiable credential. The status message descriptions <i>MUST</i> be defined in credentialSubject.statusMessages. credentialSubject.statusSize <i>MUST</i> be specified when this statusPurpose value is used.</td></tr> </table>	Value	Description		refresh service feature. This status does not invalidate the verifiable credential and is not reversible.	revocation	Used to cancel the validity of a verifiable credential . This status is not reversible.	suspension	Used to temporarily prevent the acceptance of a verifiable credential . This status is reversible.	message	Used to indicate a status message associated with a verifiable credential . The status message descriptions <i>MUST</i> be defined in credentialSubject.statusMessages . credentialSubject.statusSize <i>MUST</i> be specified when this statusPurpose value is used.
Value	Description										
	refresh service feature. This status does not invalidate the verifiable credential and is not reversible.										
revocation	Used to cancel the validity of a verifiable credential . This status is not reversible.										
suspension	Used to temporarily prevent the acceptance of a verifiable credential . This status is reversible.										
message	Used to indicate a status message associated with a verifiable credential . The status message descriptions <i>MUST</i> be defined in credentialSubject.statusMessages . credentialSubject.statusSize <i>MUST</i> be specified when this statusPurpose value is used.										
credentialSubject.encodedList	The encodedList property of the credential subject <i>MUST</i> be a Multibase-encoded base64url (with no padding) [RFC4648] representation of the GZIP-compressed [RFC1952] bitstring values for the associated range of verifiable credential status values. The uncompressed bitstring <i>MUST</i> be at least 16KB in size. The bitstring <i>MUST</i> be encoded such that the first index, with a value of zero (0), is located at the left-most bit in the bitstring and the last index, with a value of one less than the length of the bitstring (bitstring_length - 1), is located at the right-most bit in the bitstring. Further information on bitstring encoding can be found in Section 7.1 Bitstring Encoding.										
credentialSubject.ttl	The ttl is an <i>OPTIONAL</i> property that indicates the "time to live" in milliseconds before a refresh <i>SHOULD</i> be attempted. If not present, no default value is assumed. The value does not override or replace the validity period of the BitstringStatusList. Implementations that publish the status list <i>SHOULD</i> align any protocol-specific										

Property	Description
	caching information, such as the HTTP Cache-Control header, with the value in this field.

The example below demonstrates how the **BitstringStatusListEntry** is used with a **BitstringStatusListCredential** to provide the [verifier](#) with the information necessary to determine the status of a particular [verifiable credential](#).

EXAMPLE 3: Example BitstringStatusListCredential

```
{
  "@context": [
    "https://www.w3.org/ns/credentials/v2"
  ],
  "id": "https://example.com/credentials/status/3",
  "type": ["VerifiableCredential", "BitstringStatusListCredential"],
  "issuer": "did:example:12345",
  "validFrom": "2021-04-05T14:27:40Z",
  "credentialSubject": {
    "id": "https://example.com/status/3#list",
    "type": "BitstringStatusList",
    "statusPurpose": "revocation",
    "encodedList": "uH4sIAAAAAAAAAA-
3BMQEAAADCoPVPbQwfoAAAAAAAAAAAAAAAAAIC3AYbSVKsAQAAA "
  }
}
```

§ 3. Algorithms

The following section outlines the algorithms that are used to generate and validate status lists as described by this document.

If an implementation of any of the algorithms in this section processes a property defined in Section [2. Data Model](#) whose value is malformed due to not complying with associated "MUST" statements, a [MALFORMED VALUE ERROR](#) MUST be raised.

§ 3.1 Generate Algorithm

The following process, or one generating the exact output, *MUST* be followed when producing a [BitstringStatusListCredential](#). The algorithm takes a list of *issued credentials* as input and either throws an error or returns a status list credential as output.

- 1) Let *issued credentials* be a list of all issued [verifiable credentials](#).
- 2) Let *statusListCredential* be an unsigned [BitstringStatusListCredential](#) without the **encodedList** property set.
- 3) Generate a *compressed bitstring* by passing *issued credentials* to the [Bitstring Generation Algorithm](#).
- 4) Set the **encodedList** to *compressed bitstring*.
- 5) Generate a proof for the *statusListCredential* and publish it to the endpoint listed in the [verifiable credential](#).

[Issuers](#) *SHOULD* publish status list credentials in a way that can be cached and that does not track who retrieves the status list credential, such as through [Oblivious HTTP](#), a content distribution network that is not operated by the [issuer](#), or business processes for which the access logs are not accessible by data analysts or systems administrators.

§ 3.2 Validate Algorithm

The following process, or one generating the exact output, *MUST* be followed when validating a [verifiable credential](#) that is contained in a [BitstringStatusListCredential](#). The algorithm takes a status list [verifiable credential](#) as input and either throws an error or returns a status list credential as output.

- 1) Let *credentialToValidate* be a [verifiable credential](#) containing a **credentialStatus** entry that is a [BitstringStatusListEntry](#).
- 2) Let *minimumNumberOfEntries* be 131,072 unless a different lower bound is established by a specific ecosystem specification.
- 3) Let *status purpose* be the value of **statusPurpose** in the **credentialStatus** entry in the *credentialToValidate*.
- 4) Dereference the **statusListCredential** URL, and ensure that all proofs verify successfully. If the dereference fails, raise a [STATUS RETRIEVAL ERROR](#). If any of the proof verifications fail, raise a [STATUS VERIFICATION ERROR](#).
- 5) Verify that the *status purpose* is equal to a **statusPurpose** value in the *statusListCredential*. Note: The *statusListCredential* might contain multiple status purposes in a single list. If the values are not equal,

raise a [STATUS_VERIFICATION_ERROR](#).

- 6) Let *compressed bitstring* be the value of the **encodedList** property of the [BitstringStatusListCredential](#).
- 7) Let *credentialIndex* be the value of the **statusListIndex** property of the [BitstringStatusListEntry](#).
- 8) Generate a *revocation bitstring* by passing *compressed bitstring* to the [Bitstring Expansion Algorithm](#).
- 9) If the length of the *revocation bitstring* divided by **statusSize** is less than *minimumNumberOfEntries*, raise a [STATUS_LIST_LENGTH_ERROR](#).
- 10) Let *status* be the value in the *bitstring* at the position indicated by the *credentialIndex* multiplied by the *size*. If the *credentialIndex* multiplied by the *size* is a value outside of the range of the *bitstring*, a [RANGE_ERROR](#) *MUST* be raised.
- 11) Let *result* be an empty [map](#).
- 12) Set the **status** key in *result* to *status*, and set the **purpose** key in *result* to the value of **statusPurpose**.
- 13) If *status* is **0**, set the **valid** key in *result* to **true**; otherwise, set it to **false**.
- 14) If the **statusPurpose** is **message**, set the **message** key in *result* to the corresponding **message** of the **value** as indicated in the **statusMessages** array.
- 15) Return *result*.

When a **statusListCredential** URL is dereferenced, server implementations *MAY* provide a mechanism to dereference the status list as of a particular point in time. When an [issuer](#) provides such a mechanism, it enables a [verifier](#) to determine changes in status to a precision chosen by the issuer, such as hourly, daily, or weekly. If such a feature is supported, and if query parameters are supported by the URL scheme, then the name of the query parameter *MUST* be **timestamp** and the value *MUST* be a valid URL-encoded [XMLSCHEMA11-2] dateTimeStamp string value. The result of dereferencing such a timestamp-parameterized URL *MUST* be either a status list credential containing the status list as it existed at the given point in time, or a [STATUS_RETRIEVAL_ERROR](#). If the result is an error, implementations *MAY* attempt the retrieval again with a different timestamp value, or without a timestamp value, as long as the [verifier](#)'s validation rules permit such an action.

[Verifiers](#) *SHOULD* cache the retrieved status list and *SHOULD* use proxies or other mechanisms, such as [Oblivious HTTP](#), that hide retrieval behavior from the [issuer](#).

NOTE: Issuer validation is use case dependent

It is expected that a [verifier](#) will ensure that it trusts the issuer of a [verifiable credential](#), as well as the issuer of the associated [BitstringStatusListCredential](#), before using the information contained in either credential for further decision making purposes. Implementers are advised that the issuers of these credential might differ, such as when the original issuer of the [verifiable credential](#) does not maintain a record of its validity.

§ 3.3 Bitstring Generation Algorithm

The following process, or one generating the exact output, *MUST* be followed when generating a status list bitstring. The algorithm takes an *issuedCredentials* list as input and either throws an error or returns a *compressed bitstring* as output.

- 1) Let *bitstring* be a list of bits with a minimum size of 16KB, where each bit is initialized to 0 (zero).
- 2) For each value in **bitstring**, if there is a corresponding **statusListIndex** value in a credential in **issuedCredentials**, set the value to the appropriate status. The position of the value is computed as **statusListIndex** times the **statusSize**.
- 3) Generate a *compressed bitstring* by using the GZIP compression algorithm [RFC1952] on the *bitstring* and then [Multibase-encode](#) the result using base64url (with no padding).
- 4) Return the *compressed bitstring*.

§ 3.4 Bitstring Expansion Algorithm

The following process, or one generating the exact output, *MUST* be followed when expanding a compressed status list bitstring. The algorithm takes a *compressed bitstring* as input and either throws an error or returns a *uncompressed bitstring* as output.

- 1) Let *compressed bitstring* be a compressed status list bitstring.
- 2) Generate an *uncompressed bitstring* by using the [Multibase-decode](#) algorithm on the *compressed bitstring* and then expanding the output using the GZIP decompression algorithm [RFC1952].
- 3) Return the *uncompressed bitstring*.

§ 3.5 Processing Errors

The algorithms described in this specification throw specific types of errors. Implementers might find it useful to convey these errors to other libraries or software systems. This section provides specific URLs, descriptions, and error codes for the errors, such that an ecosystem implementing technologies described by this specification might interoperate more effectively when errors occur.

When exposing these errors through an HTTP interface, implementers *SHOULD* use [Problem Details for HTTP APIs](#) [RFC9457] to encode the error data structure. If [RFC9457] is used:

- The **type** value of the error object *MUST* be a URL that starts with the value **<https://www.w3.org/ns/credentials/status-list#>** and ends with the value in the section listed below.
- The **title** value *SHOULD* provide a short but specific human-readable string for the error.
- The **detail** value *SHOULD* provide a longer human-readable string for the error.
- All human-readable strings *SHOULD* be localized as described in Section 1 of [Problem Details for HTTP APIs](#), using [language negotiation](#) through the **Accept-Language** HTTP header field to select the most appropriate resources.

STATUS_RETRIEVAL_ERROR

Retrieval of the status list failed. See Section [3.2 Validate Algorithm](#).

STATUS_VERIFICATION_ERROR

Validation of the status entry failed. See Section [3.2 Validate Algorithm](#).

STATUS_LIST_LENGTH_ERROR

The status list length does not satisfy the minimum length required for herd privacy. See Section [3.2 Validate Algorithm](#).

§ 3.6 Securing Algorithms

There are multiple ways that the information in Section [2. Data Model](#) can be secured. These mechanisms are elaborated upon in the [Securing Mechanisms](#) section of the [Verifiable Credentials Data Model v2.0](#).

When securing a [verifiable credential](#) that contains a reference to a [BitstringStatusListCredential](#), implementers *SHOULD* use the same securing mechanism with the same cryptographic parameters and the same media type for both [verifiable credentials](#).

§ 4. Media Types

When dereferencing **statusListCredential**, the content of the returned **statusListCredential** might be any media type registered for the purpose of expressing a verifiable credential with one or more proofs.

For example, a verifiable credential secured with Data Integrity Proofs might have media type **application/vc**, while a verifiable credential secured with SD-JWT might have media type **application/sd-jwt**.

Some implementations might choose to support less specific media types such as **application/ld+json** or **application/json**.

When dereferencing over HTTP, the use of the [accept](#) and [content-type](#) headers, might allow some implementations to negotiate for the proof format used to secure the **statusListCredential**.

Some implementations might use the [415 Unsupported Media Type](#) status code to signal that they do not support the requested media type.

§ 5. Contexts and Vocabularies

§ 5.1 Vocabulary

The terms defined in this specification are also part of the RDF [vocabulary namespace](#) <https://www.w3.org/ns/credentials/status#>. For any **TERM**, the relevant URL is of the form **<https://www.w3.org/ns/credentials/status#TERM>**. Implementations that use RDF processing and rely on this specification *MUST* use these URLs.

When dereferencing the <https://www.w3.org/ns/credentials/status#> URL, the media type of the data that is returned depends on HTTP content negotiation. These are as follows:

Media Type	Description and Hash
application/ld+json	The vocabulary in JSON-LD format [JSON-LD11]. SHA2-256 Digest: 98b555e914aed27fe6b73dbd655a3d1103d1a26ce1ad709a38435dccb3451a68
text/turtle	The vocabulary in Turtle format [TURTLE]. SHA2-256 Digest: ad4b2142eaa57cf771d91c2b061eb4cd17d74c8e87899ea14368df14168844
text/html	The vocabulary in HTML+RDFa Format [HTML-RDFA]. SHA2-256 Digest: fa3b8be58441dfdf11f5314330d612e0a9b9e94993076f3702418c511de71174

It is possible to confirm the cryptographic digests above by running a command like the following (replacing **<MEDIA_TYPE>** and **<DOCUMENT_URL>** with the appropriate values) through a modern UNIX-like OS command line interface: **curl -sL -H "Accept: <MEDIA_TYPE>" <DOCUMENT_URL> | openssl dgst -sha256**

§ 5.2 JSON-LD context

Implementations that perform JSON-LD processing *MUST* treat the following JSON-LD context URL as already resolved, where the resolved document matches the corresponding hash value below:

Context URL and Hash

URL: <https://www.w3.org/ns/credentials/status/v1>

SHA2-256 Digest:

[fda5add353231e6a6884a46b12e6c75464281900cb348284d9c360f62381d9f7](#)

It is possible to confirm the cryptographic digests listed above by running a command like the following through a modern UNIX-like OS command line interface: **`curl -sL -H "Accept: application/ld+json" https://www.w3.org/ns/credentials/status/v1 | openssl dgst -sha256`**

The vocabulary terms that the JSON-LD contexts resolve to are in the <https://www.w3.org/ns/credentials/status#> namespace. See Section [5.1 Vocabulary](#) for further details.

NOTE

Applications or specifications may define mappings to the vocabulary URLs using their own JSON-LD contexts. For example, the JSON-LD context definitions referred to in this section are also a part of the <https://www.w3.org/ns/credentials/v2> context, defined by the [Verifiable Credentials Data Model v2.0](#) specification.

§ 6. Privacy Considerations

This section is non-normative.

This section details the general privacy considerations and specific privacy implications of deploying this specification into production environments.

Readers are urged to familiarize themselves with the general privacy advice provided in the [Privacy Considerations section of the Verifiable Credentials specification](#) before reading this section.

§ 6.1 Revocation Bitstring Length

This section is non-normative.

This document specifies a minimum revocation bitstring length of 131,072, or 16KB uncompressed. This is enough to give [holders](#) an adequate amount of group privacy if the number of verifiable credentials issued is large enough. However, if the number of issued verifiable credentials is a small population, the ability to correlate an individual increases because the number of allocated slots in the bitstring is small. Correlating this information with, for example, where the geographic request came from can also help to correlate individuals that have received a credential from the same geographic region.

§ 6.2 Unnecessary Correlation

This section is non-normative.

There are a number of global identifiers used in a status list entry, defined in Section [2.1 BitstringStatusListEntry](#), that can be used across [verifiers](#) to correlate [subjects](#). Some of the properties that can express these values are **id**, **statusListIndex**, and **statusListCredential**.

In some cases, such as when [presenting](#) a [verifiable credential](#) that contains a global identifier (such as a driver's license identification number), adding one or more global identifier(s) for status list information does not increase correlation harm, since a single globally unique identifier is all that is required for correlation.

When global identifiers are used in [presentations](#) that use [selective disclosure](#) or [unlinkable disclosure](#), they can violate privacy expectations. [Issuers](#) are urged to enable status information to be selectively disclosable/concealable when a particular [verifiable credential](#) is expected to be disclosed in a way that does not need correlation, such as when proving that an individual is above a certain age. [Verifiers](#) can require that status information be revealed in situations that require them to know the current status of a credential, and the holder might then consent or refuse to reveal that information for a given transaction. In all cases, both [issuers](#) and [verifiers](#) are urged to avoid the use of global identifiers in order to prevent correlation, unless it is required for or by a particular exchange.

For information on other types of potential correlation, readers are urged to study the Privacy Considerations section of the [Verifiable Credentials Data Model v2.0](#) specification, particularly the subsections on [Identifier-Based Correlation](#), [Signature-Based Correlation](#), [Long-Lived-Identifier-Based Correlation](#), and [Metadata-Based Correlation](#).

§ 6.3 Verifier Caching

This section is non-normative.

It is possible for [verifiers](#) to increase the privacy of the [holder](#) whose [verifiable credential](#) is being checked by caching status lists that have been fetched from remote servers. By caching the content locally, less correlatable information can be inferred from [verifier](#)-based access patterns on the status list.

§ 6.4 Content Distribution Networks

This section is non-normative.

The use of content distribution networks by [issuers](#) can increase the privacy of [holders](#) by reducing or eliminating requests for the status lists from the [issuer](#). Often, a request for a revocation list will be served by an edge device and thus be faster and reduce the load on the server as well as cloaking [verifiers](#) and [holders](#) from [issuers](#).

§ 6.5 Decoy Values

This section is non-normative.

[Issuer](#) use of decoy values in status lists has been explored as a mechanism to increase the privacy of [subjects](#). While algorithms for employing decoy values are out of scope for this specification, implementers are advised that the use of decoy values can harm privacy if the decoy values do not accurately simulate the population associated with the status list. If decoy values can be distinguished from real values, the anonymity provided in the set will be reduced by the number of decoy values that are detectable as such. The most privacy-preserving status list is one that never changes, since the behavior of the population cannot be determined if no observable events occur.

Given how difficult it is to statistically simulate status entries for a population, and that general advice cannot be given since [verifiable credentials](#) serve a broad set of use cases, implementers are advised to allocate status list entry indexes randomly, and to minimize — optimally to never — the rate at which status entries are changed. Allocation of status list entries preserves privacy best when it does not trigger any observable change of a status list.

§ 6.6 Malicious Issuers and Verifiers

This section is non-normative.

In general, the group privacy protections offered by this specification can be circumvented by malicious [issuers](#) and [verifiers](#). Its privacy benefits can only be realized when issuers and verifiers intend to avoid tracking or sharing the presentation of particular credentials.

A malicious [verifier](#) might intentionally attack group privacy by sharing information from presented credentials with a malicious [issuer](#). This sort of collusion is difficult to detect as it is typically performed via a secure communication channel between the [issuer](#) and the [verifier](#).

A malicious [issuer](#) might intentionally attack group privacy by creating a unique status list for each issued credential, in order to establish a one-to-one mapping to track when a [verifier](#) processes each mapped credential. Similarly, they could establish a one-to-one mapping by using a different cryptographic key for each issued credential that is tracked by a given status list.

This sort of collusion can be detected by [holder](#) software that serves multiple [holders](#) (e.g., a [holder](#) app that runs on a server) if it has, for example, an opt-in process that finds that some global identifier(s) used within a [verifiable credential](#) are not adequately shared by other credentials. [Holders](#) could then be warned when presenting a [verifiable credential](#) that contains some global identifier(s) that are unique to that credential. Such an opt-in service could represent some additional privacy concerns; whether this potential exposure via the [holder](#) software is justified by the awareness of possible global identifier correlation can only be evaluated by the users of such a system.

§ 6.7 Monitoring Status Lists

This section is non-normative.

Once a [verifier](#) knows of a status list and entry index that is associated with a specific [holder](#) or [subject](#), it becomes possible for that [verifier](#) to see updates to that status entry as long as the status list continues to be updated. This is useful to a [verifier](#) that needs to understand when a particular [verifiable credential](#) has changed status without asking the [issuer](#) directly for status information on the specific [verifiable credential](#) or when interacting with the [holder](#) to get the latest status information is not possible. The feature can also cause a privacy violation for the [holder](#) and/or [subject](#) if the [verifier](#) is able to perform near-real-time checks on the status of the [verifiable credential](#).

[Issuers](#) can provide a level of reprieve from this privacy concern for [holders](#) by revoking and reissuing effectively the same [verifiable credential](#) on a relatively brief timeline. For example, an [issuer](#) could automatically reissue a [verifiable credential](#) every three months and assign a new status entry index when the reissuance occurs to break any sort of long-term monitoring of a [verifiable credential](#) as it changes status.

§ 6.8 Correlation of Status Messages

This section is non-normative.

This specification provides a means by which multiple status messages can be provided for a particular entry in a status list. While this mechanism can provide more detailed information for a particular entry in the status list, that information can provide further correlation data.

For example, if each status message is associated with a step in a particular process, or more detailed information as to why a credential was revoked or suspended, then an attacker that observes the changes in the list might be able to correlate information about the population of entities in the list that could lead to privacy violations. Understanding how a population progresses through a business process, or what percentage of the population is likely to be associated with a certain status, provides additional information to an attacker. Given such information, a phishing operation could predict what the next step of a business process is and then preemptively contact an entity whose current status is known. Then, based on that information, they could attempt to phish more lucrative information from the target using data gleaned from the status list over time.

For these reasons, issuers are urged to evaluate the potential ramifications of publishing detailed status information about a particular entity, or a population, in a public manner.

§ 6.9 Alteration of Status Messages

This section is non-normative.

When a status list uses the status messages feature, it becomes possible for the [issuer](#) to increase the types of messages that are associated with the [verifiable credentials](#) it issues over time.

This feature creates a potential privacy violation where the [subject](#) or [holder](#) of the [verifiable credential](#) might be associated with additional status information that was not present when the original [verifiable credential](#) was issued. For example, initial status messages might convey "delayed" and "canceled", but additional status messages might be added by the [issuer](#) to convey "delayed due to non-payment" and "canceled due to illegal activity". This change would not be apparent to the [subject](#) or [holder](#) unless there was monitoring software operating on their behalf that would warn them that the [issuer](#) intends to expose additional information about their activity.

Holder software can provide features to [holders](#) that warn them about the level of [holder](#) and/or [subject](#) information exposure when using [verifiable credentials](#) that are associated with status messages, and warn them when the level of information exposure changes.

§ 7. Security Considerations

This section is non-normative.

There are a number of security considerations that implementers should be aware of when processing data described by this specification. Ignoring or not understanding the implications of this section can result in security vulnerabilities.

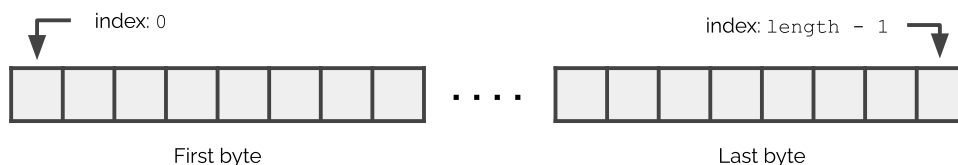
Readers are urged to familiarize themselves with the general security advice provided in the [Security Considerations section of the Verifiable Credentials specification](#) before reading this section.

While this section attempts to highlight a broad set of security considerations, it is not a complete list. Implementers are urged to seek the advice of security and cryptography professionals when implementing mission critical systems using the technology outlined in this specification.

§ 7.1 Bitstring Encoding

This section is non-normative.

It is critical that implementers pay particular attention to the way that they encode and decode bitstrings. Failure to do so can result in checking the wrong bitstring index for a given credential, leading to a misinterpretation of its present state (e.g., mistaking a revoked status for an unrevoked status). As stated in Section [2.2 BitstringStatusListCredential](#), bitstrings are encoded such that the first (zeroth) index refers to the left-most bit of the bitstring array. The diagram below demonstrates the proper layout for an uncompressed bitstring.



[Figure 2](#) A visual depiction of the bitstring layout.

For example, if a bitstring is 131,072 bits in size (16KB), the first index will be 0, and the last index will be 131,071.

§ 7.2 Validity Periods

This section is non-normative.

The [validity period](#) that an issuer might choose to express in a status list is dependent on a variety of factors including:

- How often do status values change? In real-time? Daily? Weekly? Monthly? Other?
- Is there a regulatory requirement that compels an [issuer](#) to notify a [verifier](#) that the status of a [verifiable credential](#) has changed?
- Is there a period of time after which an [issuer](#) would incur reputational damage by not notifying a [verifier](#) of a status change?
- Is there any expectation that a [verifier](#) will not accept a [verifiable credential](#) with a status list that does not expire for a period of time it deems excessive?
- Will a short validity period for a status list cause a significant network bandwidth or computing burden for an [issuer](#) or a [verifier](#)? Might this burden be mitigated by a longer validity period?

Since these factors vary with the ecosystem and credential type, there is no minimum or maximum validity period that is suggested for all status lists. [Issuers](#) will need to consider various factors that are specific to their [verifiable credential](#) types and choose validity periods that will strike the right balance in their ecosystem.

§ 8. Accessibility Considerations

This section is non-normative.

Readers are urged to familiarize themselves with the general accessibility advice provided in the [Accessibility Considerations section of the Verifiable Credentials specification](#). No further advice is provided in this specification beyond the general advice for all [verifiable credentials](#).

§ 9. Internationalization Considerations

This section is non-normative.

Readers are urged to familiarize themselves with the general internationalization advice provided in the [Internationalization Considerations section of the Verifiable Credentials specification](#). No further advice is

provided in this specification beyond the general advice for all [verifiable credentials](#).

§ A. Examples

This section is non-normative.

§ A.1 Revocable Verifiable Credential

EXAMPLE 4: A Revocable Verifiable Credential

Credential	ecdsa	eddsa	jose	cose
------------	-------	-------	------	------

```
{
  "@context": [
    "https://www.w3.org/ns/credentials/v2",
    "https://www.w3.org/ns/credentials/examples/v2"
  ],
  "id": "https://example.com/credentials/23894672394",
  "type": ["VerifiableCredential"],
  "issuer": "did:example:12345",
  "validFrom": "2021-04-05T14:27:42Z",
  "credentialStatus": {
    "id": "https://example.com/credentials/status/3#94567",
    "type": "BitstringStatusListEntry",
    "statusPurpose": "revocation",
    "statusListIndex": "94567",
    "statusListCredential": "https://example.com/credentials/status/3"
  },
  "credentialSubject": {
    "id": "did:example:6789",
    "type": "Person"
  }
}
```

§ A.2 Status List Verifiable Credential

EXAMPLE 5: A Status List Verifiable Credential

Credential	ecdsa	eddsa	jose	cose
------------	-------	-------	------	------

```
{
  "@context": [
    "https://www.w3.org/ns/credentials/v2",
    "https://www.w3.org/ns/credentials/examples/v2"
  ],
  "id": "https://example.com/credentials/status/3",
  "type": ["VerifiableCredential", "BitstringStatusListCredential"],
  "issuer": "did:example:12345",
  "validFrom": "2021-04-05T14:27:40Z",
  "credentialSubject": {
    "id": "https://example.com/status/3#list",
    "type": "BitstringStatusList",
    "statusPurpose": "revocation",
    "encodedList": "uH4sIAAAAAAAAAA-
3BMQEAAADCoPVPbQwfoAAAAAAAAAAAAAAAAAIC3AYbSVKsAQAAA "
  }
}
```

§ A.3 Multiple Status Lists in One Verifiable Credential

This specification enables an [issuer](#) to associate multiple status lists with a single [verifiable credential](#).

EXAMPLE 6: Associating multiple status lists with a single Verifiable Credential

```

{
  "@context": [
    "https://www.w3.org/ns/credentials/v2",
    "https://www.w3.org/ns/credentials/examples/v2"
  ],
  "id": "https://example.com/credentials/23894672394",
  "type": ["VerifiableCredential"],
  "issuer": "did:example:12345",
  "issuanceDate": "2021-04-05T14:27:42Z",
  // note the use of an array to represent the set of
  // status entries
  "credentialStatus": [{
    "id": "https://example.com/credentials/status/3#94567",
    "type": "BitstringStatusListEntry",
    "statusPurpose": "revocation",
    "statusListIndex": "94567",
    "statusListCredential": "https://example.com/credentials/status/3"
  }, {
    "id": "https://example.com/credentials/status/4#12345",
    "type": "BitstringStatusListEntry",
    "statusPurpose": "suspension",
    "statusListIndex": "12345",
    "statusListCredential": "https://example.com/credentials/status/4"
  }],
  "credentialSubject": {
    "id": "did:example:6789",
    "type": "Person"
  }
}

```

§ A.4 Multiple Status Entries in a Single List

It is possible for a single status list to contain multiple types of status purposes. Doing so can make the retrieval of a list slightly more efficient than fetching multiple status lists.

EXAMPLE 7: Associating multiple status entries in a single status list

```

{
  "@context": [
    "https://www.w3.org/ns/credentials/v2",
    "https://www.w3.org/ns/credentials/examples/v2"
  ],
  "id": "https://example.com/credentials/23894672394",
  "type": ["VerifiableCredential"],
  "issuer": "did:example:12345",
  "issuanceDate": "2021-04-05T14:27:42Z",
  // note the use of a single list to store multiple
  // status entries
  "credentialStatus": [{
    "id": "https://example.com/credentials/status/5#94567",
    "type": "BitstringStatusListEntry",
    "statusPurpose": "revocation",
    "statusListIndex": "94567",
    "statusListCredential": "https://example.com/credentials/status/5"
  }, {
    "id": "https://example.com/credentials/status/5#12345",
    "type": "BitstringStatusListEntry",
    "statusPurpose": "suspension",
    "statusListIndex": "12345",
    "statusListCredential": "https://example.com/credentials/status/5"
  }],
  "credentialSubject": {
    "id": "did:example:6789",
    "type": "Person"
  }
}

```

§ B. Revision History

This section contains the substantive changes that have been made to this specification over time.

Changes since the [v1.0 First Candidate Recommendation](#):

- Many editorial updates to clarify grammar, flow, and understanding of the specification contents.
- Provided final guidance on using decoy values and validity periods for status lists.
- Add a context and vocabulary section.

- Clarify that the **t**tl property doesn't override validity
- Add a **refresh** status purpose.
- Remove integer error codes from problem details

§ C. References

§ C.1 Normative references

[infra]

Infra Standard. Anne van Kesteren; Domenic Denicola. WHATWG. Living Standard. URL: <https://infra.spec.whatwg.org/>

[RDF-CONCEPTS]

Resource Description Framework (RDF): Concepts and Abstract Syntax. Graham Klyne; Jeremy Carroll. W3C. 10 February 2004. W3C Recommendation. URL: <https://www.w3.org/TR/rdf-concepts/>

[RFC1952]

GZIP file format specification version 4.3. P. Deutsch. IETF. May 1996. Informational. URL: <https://www.rfc-editor.org/rfc/rfc1952>

[RFC2119]

Key words for use in RFCs to Indicate Requirement Levels. S. Bradner. IETF. March 1997. Best Current Practice. URL: <https://www.rfc-editor.org/rfc/rfc2119>

[RFC4648]

The Base16, Base32, and Base64 Data Encodings. S. Josefsson. IETF. October 2006. Proposed Standard. URL: <https://www.rfc-editor.org/rfc/rfc4648>

[RFC8174]

Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. B. Leiba. IETF. May 2017. Best Current Practice. URL: <https://www.rfc-editor.org/rfc/rfc8174>

[RFC9457]

Problem Details for HTTP APIs. M. Nottingham; E. Wilde; S. Dalal. IETF. July 2023. Proposed Standard. URL: <https://www.rfc-editor.org/rfc/rfc9457>

[RFC9458]

Oblivious HTTP. M. Thomson; C. A. Wood. IETF. January 2024. Proposed Standard. URL: <https://www.rfc-editor.org/rfc/rfc9458>

[URL]

URL Standard. Anne van Kesteren. WHATWG. Living Standard. URL: <https://url.spec.whatwg.org/>

[VC-DATA-INTEGRITY]

[*Verifiable Credential Data Integrity 1.0*](#). Ivan Herman; Manu Sporny; Ted Thibodeau Jr; Dave Longley; Greg Bernstein. W3C. 15 May 2025. W3C Recommendation. URL: <https://www.w3.org/TR/vc-data-integrity/>

[VC-DATA-MODEL-2.0]

[*Verifiable Credentials Data Model v2.0*](#). Ivan Herman; Michael Jones; Manu Sporny; Ted Thibodeau Jr; Gabe Cohen. W3C. 15 May 2025. W3C Recommendation. URL: <https://www.w3.org/TR/vc-data-model-2.0/>

[XMLSCHEMA11-2]

[*W3C XML Schema Definition Language \(XSD\) 1.1 Part 2: Datatypes*](#). David Peterson; Sandy Gao; Ashok Malhotra; Michael Sperberg-McQueen; Henry Thompson; Paul V. Biron et al. W3C. 5 April 2012. W3C Recommendation. URL: <https://www.w3.org/TR/xmlschema11-2/>

§ C.2 Informative references

[ALLOSAUR]

[*ALLOSAUR: Accumulator with Low-Latency Oblivious Sublinear Anonymous credential Updates with Revocations*](#). Cryptology ePrint Archive. January 5th, 2024. URL: <https://eprint.iacr.org/2022/1362.pdf>

[CID]

[*Controlled Identifiers v1.0*](#). Michael Jones; Manu Sporny. W3C. 15 May 2025. W3C Recommendation. URL: <https://www.w3.org/TR/cid-1.0/>

[HTML-RDFA]

[*HTML+RDFa 1.1 - Second Edition*](#). Manu Sporny. W3C. 17 March 2015. W3C Recommendation. URL: <https://www.w3.org/TR/html-rdfa/>

[JSON-LD11]

[*JSON-LD 1.1*](#). Gregg Kellogg; Pierre-Antoine Champin; Dave Longley. W3C. 16 July 2020. W3C Recommendation. URL: <https://www.w3.org/TR/json-ld11/>

[RFC2560]

[*X.509 Internet Public Key Infrastructure Online Certificate Status Protocol - OCSP*](#). M. Myers; R. Ankney; A. Malpani; S. Galperin; C. Adams. IETF. June 1999. Proposed Standard. URL: <https://www.rfc-editor.org/rfc/rfc2560>

[RFC5280]

[*Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List \(CRL\) Profile*](#). D. Cooper; S. Santesson; S. Farrell; S. Boeyen; R. Housley; W. Polk. IETF. May 2008. Proposed Standard. URL: <https://www.rfc-editor.org/rfc/rfc5280>

[RFC7231]

[*Hypertext Transfer Protocol \(HTTP/1.1\): Semantics and Content*](#). R. Fielding, Ed.; J. Reschke, Ed. IETF. June 2014. Proposed Standard. URL: <https://httpwg.org/specs/rfc7231.html>

[RFC8932]

[Recommendations for DNS Privacy Service Operators](#). S. Dickinson; B. Overeinder; R. van Rijswijk-Deij; A. Mankin. IETF. October 2020. Best Current Practice. URL: <https://www.rfc-editor.org/rfc/rfc8932>

[TURTLE]

[RDF 1.1 Turtle](#). Eric Prud'hommeaux; Gavin Carothers. W3C. 25 February 2014. W3C Recommendation. URL: <https://www.w3.org/TR/turtle/>

